

Esempio

- Data una lista di istanze di Persona trovare i nomi delle persone hanno età minore di 26, ruolo Programmer, e ordinare i risultati per nome

```
record Persona(String nome, int eta, String ruolo) { }

List<Persona> team = List.of(new Persona("Kent", 29, "CTO"), new Persona("Luis", 25, "Programmer"),
    new Persona("Al", 25, "Programmer"), new Persona("Andrea", 26, "GroupLeader"),
    new Persona("Joe", 26, "Programmer"));

team.stream()
    .filter(p -> p.eta() < 26)
    .filter(p -> p.ruolo().equals("Programmer"))
    .sorted(Comparator.comparing(Persona::nome))
    .forEach(p -> System.out.print(p.nome() + " "));
// Output: Al Luis
```

- filter() operazione intermedia che restituisce gli elementi che soddisfano il predicato passato
- sorted() operazione intermedia stateful che restituisce uno stream che ha gli elementi ordinati in base al Comparator passato
- comparing() permette di estrarre la chiave per il confronto
- forEach() operazione terminale che esegue un'azione su ciascun elemento dello stream (su uno stream parallelo l'ordine non è garantito)

1

Prof. Tramontana - 14 Aprile 2025

Stateless—Stateful

- Le operazioni map() e filter() sono stateless, ovvero non hanno uno stato interno, prendono un elemento dello stream alla volta e forniscono un risultato
- Le operazioni come reduce(), max() accumulano un risultato. Quest'ultimo ha una dimensione limitata, indipendente da quanti elementi vi sono nello stream. Il risultato di una passata (del ciclo interno) viene dato in ingresso alla passata successiva
- Le operazioni come sorted() e distinct() devono conoscere gli altri elementi dello stream per poter eseguire, si dicono stateful

3

Prof. Tramontana - 14 Aprile 2025

Esempio

- Data una lista di istanze di Persona trovare i diversi ruoli

```
team.stream()
    .map(p -> p.ruolo())
    .distinct()
    .forEach(r -> System.out.print(r + " "));
// Output: CTO Programmer GroupLeader
```

- distinct() operazione intermedia stateful che restituisce uno stream di elementi distinti

- Data una lista di istanze di Persona trovare il nome di una che ha il ruolo Programmer

```
Optional<Persona> r = team.stream()
    .filter(p -> p.ruolo().equals("Programmer"))
    .findAny();

if (r.isPresent()) System.out.println(r.get().getNome());
// Output: Luigi
```

- findAny() (simile a findFirst()) operazione terminale che restituisce un Optional, per valutarla non è necessario esaminare tutto lo stream, si dice short-circuiting (può far sì che alcune parti non eseguano)

2

Prof. Tramontana - 14 Aprile 2025

Generare Stream: iterate()

- Gli stream possono essere generati a partire da una funzione tramite le operazioni iterate() e generate(). Queste operazioni creano stream infiniti
- iterate() restituisce un stream infinito e ordinato dato dall'esecuzione iterativa di un funzione f applicata inizialmente ad un elemento seme, quindi produce uno stream di seme, f(seme), f(f(seme)), etc.
- limit() tronca lo stream a una lunghezza pari al numero indicato

```
Stream.iterate(2, n -> n * 2)
    .limit(10)
    .forEach(System.out::println);
```

- Il codice sopra dà lo stream di 10 elementi che consiste in 2, 4, 8, ... 1024
- A ogni passo, dopo il primo, il valore in input alla funzione è il valore calcolato dalla funzione al passo precedente
- L'operazione iterate() è sequenziale

4

Prof. Tramontana - 14 Aprile 2025

Generare Stream: generate()

- Il metodo `generate()` permette di produrre uno stream infinito di valori, tramite una funzione di tipo `Supplier`, ovvero che fornisce un valore
 - `generate()` non applica una funzione a ogni nuovo valore prodotto, come invece fa `iterate()`

```
Stream.generate(() -> Math.round(Math.random()*10))
    .limit(5)
    .forEach(System.out::println);
```

- Il codice sopra genera uno stream di 5 numeri casuali di tipo `long`, ciascuno fra 0 e 10

```
Stream.generate(Math::random)
    .limit(5)
    .forEach(System.out::println);
```

- Il codice sopra genera uno stream di 5 numeri casuali di tipo `double`, ciascuno fra 0 e 1

5

Prof. Tramontana - 14 Aprile 2025

Debug Con Peek

```
List<Integer> numbers = Arrays.asList(2, 3, 4, 5);
numbers.stream()
    .map(x -> x + 17)
    .peek(x -> System.out.println("after map: " + x))
    .filter(x -> x % 2 == 0)
    .peek(x -> System.out.println("after filter: " + x))
    .limit(3)
    .peek(x -> System.out.println("after limit: " + x))
    .collect(Collectors.toList());
```

Output

```
after map: 19
after map: 20
after filter: 20
after limit: 20
after map: 21
after map: 22
after filter: 22
after limit: 22
```

7

Prof. Tramontana - 14 Aprile 2025

Debug

- Per esigenze di debug potresti voler conoscere come è fatto lo stream mano a mano che si eseguono le operazioni

```
List<Integer> numbers = Arrays.asList(2, 3, 4, 5);
numbers.stream()
    .map(x -> x + 17)
    .filter(x -> x % 2 == 0)
    .limit(3)
    .forEach(System.out::println);
```

// Output: 20 22

- Sarebbe utile capire cosa produce ciascuna operazione. Il metodo `forEach()` consuma l'elemento dello stream, quindi non si può usare

6

Prof. Tramontana - 14 Aprile 2025

Tipo IntStream

- `IntStream` rappresenta uno stream di valori `int`

```
IntStream.rangeClosed(1, 6)
    .map(x -> x*x)
    .forEach(System.out::println);
```

- Il codice sopra genera e stampa i quadrati dei numeri da 1 a 6
- `rangeClosed()` restituisce una sequenza di `int` nell'intervallo specificato (estremi inclusi) con incremento pari a 1
- `sum()` somma i valori presenti nello stream `IntStream`

```
int v = IntStream.rangeClosed(1, 5).sum();
// Output: v = 15
```

8

Prof. Tramontana - 14 Aprile 2025

Tipo IntStream

```
int result =
    Stream.of("truth", "flows", "to", "them", "sweetly", "by", "nature")
        .mapToInt(x -> x.length()).sum();
// Output: result = 31
```

- `mapToInt()` esegue la funzione passata e restituisce un `IntStream`

```
Stream<Integer> s = IntStream.rangeClosed(1, 10).boxed();
```

- `boxed()` restituisce uno `Stream` di `Integer` a partire da un `IntStream`
- Si abbia la lista che contiene istanze di `Persona`, si generi uno stream contenente i primi 4 elementi

```
List<Persona> lista;
```

```
Stream<Persona> p =
    IntStream.rangeClosed(0, 3)
        .mapToObj(i -> lista.get(i));
```

- `mapToObj()` restituisce uno stream di oggetti a partire da un `IntStream`

9

Prof. Tramontana - 14 Aprile 2025

Interfaccia Supplier<T>

- Un `Supplier<T>` è un'interfaccia funzionale che ha un singolo metodo chiamato `get()` e rappresenta una funzione che non prende in ingresso alcun parametro e restituisce un valore di tipo `T`

```
Supplier<String> sup = () -> "ciao ciao";
```

```
String s = sup.get();
```

```
// s: ciao ciao
```

- Il codice sopra implementa un `Supplier` che restituisce una stringa, quando è chiamato il metodo `get()`. Il `supplier` non tiene una stringa ma il codice per generarla
- Il metodo `iterate()` di `Stream` prende in ingresso un `Supplier`

11

Prof. Tramontana - 14 Aprile 2025

Interfaccia Function<T, R>

- L'interfaccia funzionale `Function<T, R>` definisce un metodo chiamato `apply()` che prende in ingresso un oggetto di tipo `T` e ritorna un tipo `R`

```
Function<String, Integer> stringLength = x -> x.length();
```

- In questo esempio, il metodo `apply()` ha in ingresso un parametro `String`, l'implementazione è `x.length()`, e restituisce il valore dato da `length()`
- Nel frammento seguente, `map()` usa la funzione definita sopra `stringLength` per calcolare la lunghezza di ogni parola dello stream, e quindi `reduce()` per sommare le lunghezze

```
int result =
    Stream.of("truth", "flows", "to", "them", "sweetly", "by", "nature")
        .map(stringLength)
        .reduce(0, Integer::sum);
```

```
System.out.println(result);
```

```
// Output: 31
```

10

Prof. Tramontana - 14 Aprile 2025

- Data una lista di lavoratori, ognuno con il proprio ruolo, calcolare l'ammontare da pagare a ciascuno per un mese e il totale complessivo

```
public record Persona (String nome, int eta, String ruolo) { }
```

```
public record BustaPaga(Persona dipendente, int totale) { }
```

```
public class ListaPersone {
    private List<Persona> team = List.of(new Persona("K Gray",29,"CTO"), new Persona("L Rose",25,"Programmer"),
        new Persona("Al White", 25, "Programmer"), new Persona("Joe Black", 26, "Programmer"),
        new Persona("Jill Purple", 26, "Tester"), new Persona("Bill Green", 26, "Analyst"));
```

```
    private Map<String, Integer> roleCost = Map.of("Programmer", 50, "CTO", 65, "Tester", 56, "Analyst", 54);
    private List<BustaPaga> buste;
```

```
    public void genListaPagamenti() { // per ciascuna persona del team calcola l'importo da pagare
        buste = team.stream()
            .map(pers -> new BustaPaga(pers, roleCost.getOrDefault(pers.ruolo(), 50) * 25))
            .sorted(Comparator.comparing(busta -> busta.dipendente().nome()))
            .toList();
    }
```

```
    public int calcolaSommaBuste() { // calcola il totale pagamenti
        return buste.stream()
            .map(BustaPaga::totale)
            .reduce(0, Integer::sum);
    }
```

```
    public void stampaBuste() { // stampa nome persona e costo
        buste.forEach(b -> System.out.println(b.dipendente().nome() + "\t" + b.totale()));
    }
```

```
    public void stampaRuoliPersone() { // stampa il ruolo e l'elenco delle persone nel ruolo
        roleCost.keySet().stream()
            .sorted()
            .peek(role -> System.out.print("\nRuolo " + role + ": "))
            .forEach(role -> team.stream()
                .filter(pers -> pers.ruolo().equals(role))
                .sorted(Comparator.comparing(Persona::nome))
                .forEach(pers -> System.out.print(pers.nome() + ", ")));
    }
```

}

12

Prof. Tramontana - 14 Aprile 2025